

GWEB

Merchant Onboarding & Underwriting Intake Layer

Intern Coding Capability Assessment

AWS Lambda-only backend | Amazon S3 document storage | Amazon DynamoDB metadata

Why this task matters

This is not a throwaway coding exercise. Strong work may be integrated into the GWEB merchant onboarding platform. If only parts meet production standards, GWEB may selectively reuse or adapt those components. Design your submission as if another engineer must maintain it after handoff.

Assessment type	Full-stack / backend-heavy implementation test
Primary domain	Merchant onboarding, KYB/KYC-style intake, document collection, underwriting support
Compute constraint	AWS Lambda only for backend application compute
Hard latency constraint	Every synchronous request / external-data handshake must complete within 45 seconds
Expected outcome	Runnable prototype, documented architecture, tests, and deployment instructions

1. Objective

Build a merchant onboarding processing layer that collects applicant and business information, verifies completeness, accepts supporting documents, classifies the merchant's business type using Merchant Category Codes (MCCs), and produces an explainable underwriting-assistance result. The prototype should be designed for later integration with GWEB's payment-processor onboarding workflow.

Important boundary

This system is an intake and decision-support prototype. It must not pretend to perform authoritative identity verification, credit decisions, sanctions screening, or final merchant approval unless a real provider is explicitly integrated. Mock external providers where credentials are unavailable.

2. Required User Journey

- 1. Applicant account / session:** Start or resume an application using a unique application ID. Preserve state so a partially completed application can be resumed.
- 2. Individual / controller information:** Collect identity and contact data for the applicant and relevant beneficial owners / controlling persons.
- 3. Corporate verification intake:** Collect legal entity information, registration identifiers, ownership information, operating addresses, and business profile.
- 4. Document upload:** Collect registration evidence, IDs, licenses (when applicable), bank evidence, and optional processor statements.
- 5. Business classification:** Applicant self-selects an industry / business activity. System proposes an MCC and risk tags; applicant confirms or corrects it.
- 6. Rate & business evaluation:** If a current processing statement is uploaded, extract or accept current rates/fees and provide an AI-assisted comparison / summary. Analyze business profile and highlight information that may require manual review.
- 7. Review & submit:** Show all entered data, missing items, warnings, document status, proposed MCC, and evaluation summary before final submission.
- 8. Internal review payload:** Produce a normalized record that GWEB can later map to one or more processor/provider onboarding APIs.

3. Minimum Data Model

3.1 Individual / control person

- Legal first name, middle name (optional), legal last name
- Date of birth
- Residential address: line 1, line 2, city, state/province, postal code, country
- Email and phone
- Role/title in the business
- Ownership percentage, if applicable
- Government-issued identification type + last four / masked identifier in metadata
- Consent / attestation timestamps and version of terms accepted

3.2 Business / legal entity

- Legal business name and DBA / trade name
- Entity type (LLC, corporation, partnership, sole proprietor, nonprofit, other)
- Formation country and state/province

- Registration identifier: EIN / federal tax ID where applicable, state UBI or local business registration number, or jurisdiction-equivalent identifier
- Registered address and operating address
- Website / URL and customer-facing business description
- Business start date
- Expected annual card volume, average ticket, highest ticket, monthly transaction count, card-present vs card-not-present mix, e-commerce percentage
- Ownership / beneficial-owner structure and controlling person(s)
- Requested settlement bank account metadata (do not store raw online-banking credentials)
- Existing payment processor, if any, and optional processing history

4. Required Documents & Upload Design

The application must support document types, upload status, versioning, review status, and a secure object reference. Actual files belong in Amazon S3; DynamoDB stores metadata and state.

Document	Required / conditional	Examples	Metadata to retain
Government ID	Required for relevant individual(s)	Driver license, passport, national ID	type, owner, S3 key, checksum, status, uploaded_at, expires_at
Business registration	Required	Articles/certificate, state registration, UBI/local registration evidence	jurisdiction, identifier, S3 key, status
Business license	Conditional	Local/professional/industry license	license type, issuer, number (masked where appropriate), expiry
Bank evidence	Required	Voided check or recent bank statement	account holder, bank name, last4, statement date, S3 key
Processing statement	Optional but used for rate evaluation	Recent merchant processing statement	processor, period, extracted metrics, confidence
Additional underwriting evidence	Conditional	Invoices, supplier agreements, fulfillment evidence, website screenshots	document category, reason requested, review status

4.1 Upload requirements

- Use pre-signed S3 URLs so large files upload directly from the client to S3; do not proxy document bodies through Lambda unless justified.
- Allow PDF, JPG/JPEG, and PNG at minimum. Validate MIME type, extension, size, and file signature where practical.
- Use unique, non-guessable S3 object keys. Never expose an unrestricted public bucket.
- Store SHA-256 checksum (or equivalent) and upload timestamp to support integrity checks and idempotency.
- Record document lifecycle: REQUESTED -> UPLOADING -> RECEIVED -> PROCESSING -> ACCEPTED / NEEDS_REVIEW / REJECTED.
- Sensitive data must not be written to application logs. Redact or mask IDs, tax identifiers, and bank account numbers in logs and UI after capture.

5. MCC Classification & Risk Policy

Implement a searchable Merchant Category Code catalog. MCC is a four-digit merchant classification used by the card ecosystem. The prototype should keep MCC classification separate from GWEB/provider risk policy so rules can be changed without rewriting the catalog.

Required behavior

The applicant chooses an industry/business activity first. The system proposes one or more likely MCCs based on the business description and selected industry. Show code + description + confidence/reason. The applicant confirms a choice, but ambiguous or sensitive categories must be marked for manual review.

- Import or seed a real MCC dataset rather than inventing codes. Include a script or documented process to refresh the dataset.
- Model risk as configurable policy attributes (for example: standard, enhanced-review, restricted/unsupported) rather than claiming an MCC is universally “high risk.”
- Support provider-specific overrides, because acquirers/processors can apply different underwriting policies to the same MCC.
- At minimum demonstrate enhanced-review logic for financial/quasi-cash/security-related activities. Mastercard documentation, for example, calls out MCC 6012, 6051, and 6211 for certain specialized purchase types.
- Persist both applicant-selected activity and system-proposed MCC so reviewers can inspect mismatches.

- Never auto-approve a merchant solely because an AI model labels it low risk.

6. AI-Assisted Evaluation Layer

Build an AI-assisted analysis module. It may use a mock adapter if no external model/API credentials are supplied, but the interface and failure behavior must be production-minded.

- Business profile summary: what the merchant sells, sales channel, geography, fulfillment model, recurring/subscription behavior, and customer type.
- MCC suggestion: return candidate MCC(s), confidence, and short explanation based on structured fields + business description.
- Document / statement extraction: parse a processing statement (or fixture) into normalized values such as processor, monthly volume, discount rate/effective rate, transaction fees, monthly fees, chargeback-related fees if visible, and statement period.
- Current-rate analysis: calculate a transparent effective processing cost where enough data exists. Separate deterministic arithmetic from AI-generated commentary.
- Risk signals: identify contradictions, missing evidence, unusually high ticket size, business-description/MCC mismatch, incomplete ownership data, or regulated-license claims without uploaded evidence.
- Output must be explainable: every warning should cite which input field/document caused it.
- AI output must be treated as untrusted structured input. Validate JSON/schema, impose token/size limits, handle timeouts, and fall back safely.

7. AWS Architecture - Hard Constraints

NON-NEGOTIABLE

All backend application compute must run on AWS Lambda. Do not deploy EC2, ECS, EKS, Fargate, App Runner, containers as persistent services, or any always-on application server. API Gateway and managed AWS data/security services are allowed as supporting infrastructure; they are not substitutes for Lambda compute.

Layer	Required / allowed	Expectation
Client	Web UI of your choice	Multi-step form, upload progress, resume state, validation and final review.
API ingress	Amazon API Gateway recommended	REST or HTTP API in front of Lambda; document routes and auth assumptions.
Application compute	AWS Lambda - required	Stateless handlers, bounded execution, retries and idempotency.
Metadata / workflow state	Amazon DynamoDB - required	Application, person/entity, document metadata, status, MCC/evaluation records.
Document/object storage	Amazon S3 - required	Private objects; direct pre-signed upload; lifecycle/versioning approach documented.
AI / external APIs	Lambda adapter	Strict timeout budget; schema validation; mockable integration.
Secrets	AWS Secrets Manager or SSM Parameter Store recommended	No API keys or credentials in source code.
Observability	CloudWatch	Structured logs, correlation/application ID, metrics, failures; no sensitive raw values.

7.1 The 45-second rule

This assessment imposes a stricter application rule than Lambda itself: every synchronous API operation, including handshakes to S3 or external services, must return within 45 seconds.

- Set explicit downstream timeouts. A request must never wait indefinitely for AI, storage, or a third-party provider.
- Design for an internal deadline below the hard limit (recommended target: ≤ 35 seconds) so cleanup and response serialization have headroom.
- If a workflow can exceed the budget, convert it to an asynchronous state transition (for example, RECEIVED -> PROCESSING) and let the client poll a status endpoint. Any individual synchronous poll/operation is still subject to the 45-second rule.
- Do not hold Lambda open while a user uploads a large file. Use a pre-signed S3 upload URL.
- Use bounded retries with backoff/jitter only when the remaining deadline permits.
- Return a clear timeout/error state and preserve enough workflow state for safe retry.
- Demonstrate at least one test where a mocked external dependency hangs or responds slowly and your function exits safely before the hard limit.

8. Suggested API Surface

Exact naming is flexible, but your API should cover the following behaviors:

- POST /applications - create application
- GET /applications/{id} - retrieve normalized state
- PATCH /applications/{id}/applicant - individual/control-person fields
- PATCH /applications/{id}/business - entity/business fields
- POST /applications/{id}/documents/presign - create restricted upload URL
- POST /applications/{id}/documents/{documentId}/complete - confirm upload and checksum/metadata
- GET /mcc?query=... - search MCC catalog

- POST /applications/{id}/classify - business/MCC suggestion
- POST /applications/{id}/evaluate - underwriting/rate analysis request
- GET /applications/{id}/evaluation - retrieve evaluation state/result
- POST /applications/{id}/submit - validate required fields and lock/submit a version for review

9. DynamoDB Expectations

- Choose and document your table strategy (single-table or multiple-table). We care more about access patterns and consistency than a fashionable schema.
- Support idempotent writes using request IDs / conditional writes where appropriate.
- Use optimistic concurrency or version fields to prevent silent overwrites.
- Document at least these access patterns: load application; list people/owners; list documents; fetch current evaluation; find submission status; query MCC catalog (DynamoDB or packaged/static data is acceptable if justified).
- Do not place large document blobs or full PDFs/images in DynamoDB.

10. Security & Privacy Requirements

- Principle of least privilege IAM. A Lambda should receive only the S3/DynamoDB actions it needs.
- Private S3 bucket; block public access. Use encryption at rest. Explain whether/how you would use KMS in production.
- All transport must use HTTPS/TLS.
- Input validation on every API boundary. Protect against oversized bodies, invalid content types, path manipulation, malformed JSON, and duplicate/replayed submissions.
- Do not accept or store card PAN/CVV in this assignment. This is merchant onboarding, not card-payment capture.
- Mask tax IDs, personal identifiers, and bank account data in responses where the full value is no longer needed.
- Document retention/deletion approach: how an abandoned application and associated S3 objects would eventually be removed.
- Audit fields: created_at, updated_at, actor/source, request/correlation ID, and material status changes.
- Do not log uploaded document contents, identity images, bank statements, raw secret values, or AI prompts that contain unnecessary personal data.

11. Engineering Quality Requirements

- Language/framework: your choice, but it must be suitable for AWS Lambda and reproducibly deployable.
- Infrastructure as Code is required: AWS SAM, Serverless Framework, AWS CDK, Terraform, or equivalent.
- Configuration must be environment-based. No hard-coded bucket names, regions, API keys, or secrets.
- Use typed models/schema validation where the language supports it.
- Provide unit tests for validation, MCC/risk mapping, rate arithmetic, and Lambda handlers/service logic.
- Provide at least one integration or local end-to-end test covering create application -> update business -> pre-sign upload -> mock upload completion -> classify -> evaluate -> submit.
- Provide fixture data only; do not commit real applicant PII or real bank statements.
- Include a README with local setup, deployment, API examples, assumptions, known gaps, and cleanup commands.
- Code should be organized so storage, external providers, AI, and business policy are replaceable adapters/modules rather than embedded in one large handler.

12. Deliverables

Source repository: Application code, tests, infrastructure code, migrations/seeding/import utilities, and sample fixtures.

Runnable demo: Deployed test endpoint or a reproducible local Lambda/API simulation plus deployment instructions.

Architecture document: One diagram and short explanation of request flow, S3 upload path, DynamoDB state, AI/external calls, timeouts, retries, and failure states.

API documentation: OpenAPI/Swagger, Postman collection, or equivalent endpoint specification.

MCC implementation: Seed/import source, search behavior, self-selected business type mapping, proposed MCC workflow, and configurable risk-policy example.

Test evidence: Automated test output plus a short note demonstrating the <=45-second timeout behavior.

Security note: Threat/abuse considerations, IAM approach, logging/redaction decisions, and production improvements.

Demo walkthrough: A concise walkthrough showing a merchant application from start through submission/review payload.

13. Acceptance Criteria (Must Pass)

- **PASS:** Backend application compute is Lambda-only.
- **PASS:** S3 is used for uploaded documents and DynamoDB for metadata/workflow state.
- **PASS:** No synchronous request can exceed the 45-second application deadline; slow dependency behavior is demonstrated.
- **PASS:** Applicant + business + ownership/controller data can be entered, validated, saved, resumed, and reviewed.

- **PASS:** Documents upload securely with metadata and lifecycle state.
- **PASS:** Business type can be self-selected; the system provides an MCC suggestion from a real MCC catalog.
- **PASS:** Risk policy is configurable and clearly separated from MCC taxonomy.
- **PASS:** AI/rate evaluation returns structured, explainable output and fails safely.
- **PASS:** Submission blocks when required items are missing and produces a normalized internal review payload when complete.
- **PASS:** Tests, documentation, deployment instructions, and no-real-PII fixtures are included.

14. Scoring Rubric (100 points)

Category	Points	What earns strong credit	Score
Functional completeness	25	End-to-end workflow works; form/state/documents/MCC/evaluation/submission are coherent.	_____
Architecture & Lambda discipline	20	Lambda-only compute, sensible API/S3/DynamoDB design, idempotency, state transitions, replaceable adapters.	_____
Security & privacy	15	Least privilege, private S3, validation, secrets handling, masking/redaction, no payment-card capture.	_____
45-second reliability	15	Explicit timeout budget, slow-dependency handling, bounded retries, async fallback when necessary, tests.	_____
Code quality & testing	15	Readable modular code, schemas/types, meaningful unit/integration tests, good errors and observability.	_____
MCC / underwriting logic	10	Real MCC source, good search/suggestion UX, configurable policy, explainable flags and no fake auto-approval.	_____

15. Bonus / Differentiators (do not replace core requirements)

- Accessible and polished form UX with save/resume and clear error recovery.
- Document OCR/extraction pipeline with confidence scores and manual correction UI.
- Rule-versioning for processor-specific underwriting policies.
- Duplicate-application detection using non-sensitive fingerprints / deterministic business identifiers.
- Event-driven asynchronous evaluation using Lambda-compatible AWS managed services while preserving the 45-second synchronous boundary.
- Rate comparison that cleanly separates known statement values, calculated metrics, and model-generated interpretation.
- A reviewer/admin screen showing risk flags, document status, data provenance, and reason codes.

16. What We Are Evaluating

We are evaluating how you reason about a real production-shaped problem: data modeling, API boundaries, security, failure handling, cloud constraints, third-party integrations, and maintainability. A smaller implementation that is reliable and well-structured is preferable to a flashy demo that hides errors or ignores constraints.

Production mindset

Assume this code may be cherry-picked into a real onboarding platform. Make every important decision easy for another engineer to understand, test, replace, and audit.

17. Reference Starting Points

Visa Merchant Data Standards Manual (April 2026): Visa public MCC listing and definitions.

<https://usa.visa.com/content/dam/VCOM/download/merchants/visa-merchant-data-standards-manual.pdf>

Mastercard Rules / Compliance Programs: Use Mastercard official rules as the source of truth for current network requirements. <https://www.mastercard.com/us/en/business/support/rules.html>

Mastercard Gateway documentation: Examples show MCC as a 4-digit business classification and specialized purchase-type handling for certain MCCs, including 6012, 6051 and 6211. Use current official documentation rather than copying a stale third-party “high-risk MCC” list.

AWS Lambda, S3, DynamoDB and API Gateway documentation: Use current AWS documentation for service limits, IAM, presigned uploads, retries, timeouts, and deployment behavior.

End of Assessment